

Revisiting the Comparison of GRU4Rec Implementations

A Re-Evaluation of RecPack and the Official Implementation
Under Controlled Experimental Settings

Liam Leirs

s0210395



Master's Thesis
Department of Computer Science
June 2026

Abstract

Session-based recommender systems have become increasingly important in scenarios where long-term user profiles are unavailable or unreliable. GRU4Rec is one of the earliest and most influential neural approaches for session-based recommendation and has been reimplemented in several modern frameworks, including RecPack. Previous comparison studies reported substantial performance and efficiency differences between these implementations, suggesting that the original GRU4Rec implementation significantly outperforms RecPack.

This work investigates whether these discrepancies originate from the recommendation architecture itself or from implementation-level differences. A detailed analysis of both codebases revealed multiple differences related to embedding configuration, candidate scoring, batching strategy, dropout handling, ranking procedures, and hyperparameter optimization. Based on these findings, several modifications were introduced to the RecPack implementation in order to more closely align it with the original GRU4Rec design.

Controlled experiments and ablation studies were conducted on multiple public datasets using both Cross-Entropy and BPR-max objectives. The results show that implementation details can substantially influence both recommendation performance and computational efficiency. In particular, enabling a no-embedding configuration significantly improves RecPack performance, especially for BPR-max, while ranking procedures under tied prediction scores can drastically alter reported evaluation metrics and affect hyperparameter selection.

After aligning important implementation details and independently optimizing both implementations, the performance gap between RecPack and the original GRU4Rec implementation becomes substantially smaller on several datasets, particularly when using the Cross-Entropy objective. However, notable performance and runtime differences remain on large-scale datasets, where implementation choices related to candidate scoring and batching become increasingly important.

Overall, the results suggest that a substantial portion of the performance gap reported in previous comparison studies can be attributed to implementation-level differences and experimental methodology rather than to the underlying recommendation architecture alone. These findings highlight the importance of implementation transparency, fair evaluation procedures, and independent hyperparameter optimization in recommender system research.

Contents

1	Introduction	4
1.1	Problem Statement	5
1.2	Research Questions	5
1.3	Contributions	6
2	Related Work	7
2.1	Session-Based Recommendation	7
2.1.1	Problem Definition	7
2.1.2	Model-Free Approaches	7
2.1.3	Model-Based Approaches	8
2.1.4	Deep Learning-Based Approaches	8
2.2	GRU4Rec	9
2.3	Comparison of GRU4Rec Implementations	10
2.4	Reproducibility and Fair Comparison in Machine Learning	10
3	Methodology	12
3.1	GRU4Rec	12
3.1.1	Session Representation	12
3.1.2	Architecture	13
3.1.3	Session-Parallel Mini-Batches	13
3.1.4	Loss Functions	14
3.1.5	Candidate Sampling During Training	16
3.1.6	Ranking Behavior	16
3.2	Official GRU4Rec Implementation	17
3.3	RecPack Implementation	18
4	Analysis of Implementation Differences	20
4.1	Training, Validation and Prediction	20
4.2	Training-Time Differences	21
4.2.1	Candidate Scoring Strategy	21
4.2.2	Batching Strategy	23
4.3	Model Architecture Differences	24
4.3.1	Embedding Configuration	24
4.3.2	Dropout Configuration	25
4.3.3	Weight Initialization	25
4.4	Evaluation Differences	26
4.4.1	Ranking Under Tied Scores	26
4.5	Computational Considerations	26
4.6	Summary	27

5	Experimental Setup	28
5.1	Datasets and Preprocessing	28
5.1.1	Datasets Used in the Original Comparison	28
5.1.2	Datasets Used in This Study	28
5.2	Evaluation Protocol	30
5.3	Hyperparameter Tuning	30
5.4	Training Setup	30
6	Experiments	32
6.1	Reproduction of the Original Comparison	32
6.2	Ablation Studies	33
6.3	Runtime Analysis	33
6.4	Evaluation on Additional Datasets	33
7	Results	35
7.1	Reproduction of the Original Experiments	35
7.2	Ablation Study	36
7.2.1	Effect of Removing Embeddings	36
7.2.2	Effect of Ranking Strategy	37
7.2.3	Impact of Decoupled Dropout	38
7.3	Runtime Analysis	39
7.3.1	Training versus Evaluation Runtime	39
7.3.2	Impact of Static Batching	39
7.4	Evaluation on Additional Datasets	40
7.4.1	YooChoose	41
7.4.2	LastFM1K	41
7.4.3	Instacart	41
7.4.4	MovieLens	42
7.4.5	Taobao	42
7.5	Summary of Findings	43
8	Discussion	45
8.1	Revisiting the Original Comparison	45
8.2	Why Does the Gap Persist for BPR-max?	46
8.3	Generalization Across Datasets	46
8.4	Implications for Reproducibility	46
8.5	Research Questions Revisited	47
8.6	Limitations	48
9	Conclusion	49
	Appendices	51
A	Best Hyperparameter Configurations	52

Chapter 1

Introduction

Recommender systems have become an essential component of modern online platforms, helping users navigate large item catalogs and discover relevant content. They are widely deployed in domains such as e-commerce, music streaming, video recommendation, and online advertising, where the number of available items often exceeds what users can reasonably explore manually [24]. Over the past decades, a wide range of recommendation approaches have been proposed, ranging from traditional neighborhood-based methods and matrix factorization techniques to more recent deep learning-based models [24, 29].

Traditional recommender systems typically rely on long-term user profiles constructed from historical interactions. While this approach has proven effective in many settings, it assumes that sufficient user history is available and that user preferences remain relatively stable over time. In practice, these assumptions do not always hold. Many real-world applications involve anonymous users, short interaction sessions, or rapidly changing user interests, making long-term profiles less informative [15].

To address these limitations, session-based recommender systems have gained increasing attention in recent years. Rather than relying on historical user profiles, session-based recommenders model only the sequence of interactions observed within a single session and use this information to predict the next item a user is likely to interact with [12]. This setting is particularly relevant in e-commerce platforms, where users often browse anonymously, and in content recommendation scenarios where recent actions provide a stronger indication of current intent than older interactions [21].

One of the most influential neural approaches for session-based recommendation is GRU4Rec, introduced by Hidasi et al. [12]. GRU4Rec is based on Gated Recurrent Units (GRUs), a type of recurrent neural network architecture designed to model sequential dependencies while mitigating issues such as vanishing gradients [6]. By treating user interactions within a session as a sequence, GRU4Rec learns to predict the next item that is most likely to occur. The model has served as a strong baseline for session-based recommendation research and has inspired numerous subsequent approaches [15, 21].

Since its introduction, several implementations of GRU4Rec have been developed. The official implementation was released in Theano [8] by the original authors [12], while later reimplementations were created in more modern frameworks such as PyTorch [18]. One widely used implementation is provided by the RecPack

framework [17], an open-source recommendation library that includes GRU4Rec [12] alongside many other recommendation algorithms. Due to its modular design and PyTorch foundation, RecPack is often used for experimentation and educational purposes.

However, differences between implementations can significantly affect reported performance. In a later comparison study, the original authors of GRU4Rec evaluated several third-party implementations and concluded that many of them, including RecPack, performed substantially worse than the official implementation [10]. The study attributed these discrepancies primarily to implementation differences and unsupported algorithmic features. Such findings raise important questions regarding reproducibility, fairness of experimental comparisons, and the extent to which reported performance differences originate from the recommendation algorithm itself rather than from implementation choices.

This thesis revisits the comparison between the official GRU4Rec implementation [12, 10] and RecPack [17]. Through detailed code analysis and controlled experimentation, the study investigates whether previously reported performance differences are truly caused by implementation errors or whether they can instead be explained by differences in hyperparameter tuning, evaluation methodology, and experimental setup. In addition, the work examines how specific implementation choices affect both recommendation quality and computational efficiency across multiple datasets.

1.1 Problem Statement

Although previous work has compared the official GRU4Rec implementation with several third-party implementations, including RecPack [10], the reported performance differences may be influenced by factors beyond the underlying recommendation model. Differences in hyperparameter tuning, evaluation procedures, implementation details, and supported configurations can all substantially affect the reported results.

As a consequence, it remains unclear whether the observed performance gaps genuinely reflect differences between implementations or whether they are partially caused by methodological inconsistencies in the experimental setup. This ambiguity makes it difficult to draw reliable conclusions regarding the quality of third-party implementations and the reproducibility of published results.

This thesis therefore re-evaluates the comparison between GRU4Rec and RecPack [10] under controlled conditions. By applying consistent preprocessing, evaluation procedures, and hyperparameter optimization to both implementations, the study aims to determine which performance differences are attributable to implementation choices and which are caused by experimental factors.

1.2 Research Questions

This thesis aims to answer the following research questions:

- RQ1: To what extent are the reported performance differences between the official GRU4Rec implementation and the RecPack implementation caused by implementation differences rather than differences in experimental setup and hyperparameter tuning?

- RQ2: Does the experimental setup used in the original comparison lead to a fair comparison between the official GRU4Rec implementation and RecPack?
- RQ3: How large is the performance difference between the official GRU4Rec implementation and RecPack across different datasets, and is this difference consistent across datasets?

1.3 Contributions

The main contributions of this thesis are as follows:

- A detailed implementation-level analysis of the official GRU4Rec [12] implementation and the RecPack [17] reimplementations.
- A controlled comparison between both implementations across multiple datasets using consistent evaluation and hyperparameter optimization procedures.
- Several modifications to the RecPack [17] implementation that enable a more faithful reproduction of the original GRU4Rec [12] design choices.
- Ablation studies investigating the impact of embedding configuration, ranking strategies, and dropout parameterization on recommendation performance.
- An empirical assessment of the fairness and reproducibility of previously reported implementation comparisons.

Chapter 2

Related Work

2.1 Session-Based Recommendation

2.1.1 Problem Definition

Traditional recommender systems typically rely on long-term user profiles to generate recommendations. These systems use historical interactions such as purchases, page views, or clicks to model user preferences over time [24, 29]. While such approaches can generate accurate recommendations when sufficient user history is available, they do not explicitly account for the fact that user preferences may change over time. Furthermore, in many real-world scenarios, long-term user histories are not available, for example when users browse anonymously or when new users have very limited interaction history [14, 21].

To address these limitations, session-based recommender systems were introduced [14, 16]. Instead of relying on long-term user profiles, session-based recommenders focus only on a short sequence of recent interactions, referred to as a session. A session typically represents a sequence of user interactions within a limited time window, such as a browsing session on an e-commerce website. By focusing on recent interactions, session-based recommenders aim to capture the user’s short-term preferences and current intent rather than long-term preferences [14, 21].

In session-based recommendation, the task is usually formulated as a next-item prediction problem [14]. Given a session consisting of a sequence of items

$$S = (i_1, i_2, \dots, i_t)$$

the goal is to predict the next item i_{t+1} . The recommender system therefore models the sequential dependencies between items within a session and produces a ranked list of candidate items that are likely to be interacted with next.

2.1.2 Model-Free Approaches

Model-free approaches to session-based recommendation rely on data mining and similarity-based techniques to extract patterns directly from session data without training an explicit predictive model [14, 16].

One common approach is based on frequent pattern mining and association rule learning [4]. Association rules capture relationships between items that frequently occur together within sessions. In the context of session-based recommendation,

pairwise association rules are often used, where the occurrence of one item increases the likelihood of recommending another item [4].

Another widely used approach is based on similarity methods such as Item-KNN [25]. In these methods, similarities between items are computed based on their co-occurrence within sessions. Recommendations are then generated by selecting items that are most similar to items already present in the current session [16].

While model-free methods are relatively simple and computationally efficient, they primarily rely on item co-occurrence and similarity information and do not explicitly model the full sequential structure of sessions. As a result, these methods are limited in their ability to capture complex sequential patterns in user behavior [16].

2.1.3 Model-Based Approaches

Model-based approaches aim to explicitly model the sequential nature of sessions and use this model to predict the next item in a session [14]. One of the most common model-based approaches for session-based recommendation is based on Markov chains. In these models, the probability of the next item depends on the previous item or a sequence of previous items [26, 21].

An alternative formulation models the recommendation problem as a Markov Decision Process (MDP) [26]. In this formulation, states correspond to sequences of user interactions, and actions correspond to recommending items. The transition function models the probability of the user selecting the recommended item or another item, and rewards encode the utility of the resulting state.

Although this approach is theoretically appealing, the state space can become extremely large when many items and long sequences are considered [26]. Despite these limitations, model-based approaches such as Markov models and MDP-based methods highlight the importance of modeling sequential dependencies in session-based recommendation.

2.1.4 Deep Learning-Based Approaches

To overcome the limitations of traditional model-based approaches such as Markov chains, deep learning methods were introduced for session-based recommendation [29, 14]. Deep learning models are particularly well suited for sequential data, as they can learn complex patterns and long-term dependencies directly from interaction sequences without explicitly defining transition probabilities or state representations.

Recurrent Neural Networks (RNNs) are a natural choice for modeling sequential data, as they maintain a hidden state that captures information about previous elements in a sequence. However, standard RNNs suffer from the vanishing gradient problem, making it difficult to learn long-term dependencies [6].

To address this issue, more advanced recurrent architectures such as Long Short-Term Memory (LSTM) networks and Gated Recurrent Units (GRUs) were introduced [6]. GRUs are a simplified version of LSTMs that use gating mechanisms to control the flow of information through the network, allowing them to capture longer dependencies while being computationally more efficient [6]. One of the first and most influential deep learning approaches for session-based recommendation

was GRU4Rec [12]. Since GRU4Rec forms the basis of this thesis, it is discussed in more detail in the next section.

A more comprehensive overview of session-based recommender systems can be found in the survey papers by [14, 16].

2.2 GRU4Rec

Recurrent Neural Networks (RNNs) have been widely used in domains such as natural language processing due to their ability to model variable-length sequential data and capture temporal dependencies within sequences [6]. Since session-based recommendation can be formulated as predicting the next item in a sequence of user interactions, RNNs are a natural choice for this task. GRU4Rec [12] was one of the first models to successfully apply recurrent neural networks to session-based recommendation and is generally considered a milestone in the development of neural session-based recommenders [14, 16].

In GRU4Rec, a session is represented as a sequence of items that is processed sequentially by a recurrent neural network [12]. At each time step, the current item is provided as input to the network, and the hidden state is updated to reflect the current state of the session. This hidden state represents the user’s short-term preferences within the session. Based on this hidden state, the model produces scores for candidate items, which are then used to rank items according to their likelihood of being the next item in the session [12].

Standard RNN training procedures are not well suited for session data, as sessions vary significantly in length and using a sliding window approach would break the sequential nature of sessions. To address this issue, the authors introduced session-parallel mini-batches [12]. In this training scheme, multiple sessions are processed in parallel, where each position in the mini-batch corresponds to a different session. When a session ends, it is replaced by a new session in the mini-batch. This allows efficient training while preserving the sequential structure of sessions [12].

Another challenge in recommender systems is the large number of items in the catalog. Computing scores for all items at each time step can be computationally expensive. Therefore, GRU4Rec uses a sampling strategy where only a subset of negative items is considered during training [12]. In practice, items appearing in other sessions within the same mini-batch are used as implicit negative samples, which significantly reduces computational cost while still providing informative negative examples for training [12, 11].

Since the goal of session-based recommendation is to produce a ranking of items rather than to perform classification, the authors proposed using pairwise ranking loss functions instead of pointwise classification losses such as Cross-Entropy [12]. GRU4Rec uses ranking losses such as Bayesian Personalized Ranking (BPR) [22] and the TOP1 loss proposed by the authors [12]. The authors later introduced improved loss functions such as BPR-max and TOP1-max [11], which focus learning on difficult negative samples and improve training efficiency.

GRU4Rec was shown to outperform traditional baselines such as Item-KNN and popularity-based recommendation in terms of Recall@20 and MRR@20 on the YooChoose dataset [12, 5]. Due to its strong performance and its ability to model

sequential user behavior, GRU4Rec became one of the most influential baseline models for session-based recommendation and is widely used as a benchmark in later research [14, 16, 21].

2.3 Comparison of GRU4Rec Implementations

After the publication of GRU4Rec, several third-party implementations of the algorithm were developed in different frameworks [10]. The original authors later published a study comparing their official implementation with several third-party implementations [10]. The goal of this study was to investigate whether these independent implementations reproduced the performance of the original model.

In their comparison, the authors evaluated multiple implementations on several publicly available datasets using identical preprocessing and evaluation protocols [10]. In addition to directly evaluating the third-party implementations, the authors also inspected their code and applied minor and major fixes where necessary in order to ensure that the implementations were functioning as intended [10].

Despite these corrections, the authors reported that their official implementation consistently outperformed the third-party implementations in terms of Recall@20 and MRR@20 across multiple datasets [10]. Based on these results, they concluded that many third-party implementations of GRU4Rec contained implementation mistakes or missing features that led to significantly lower performance [10].

These findings highlight the importance of reproducibility and fair experimental comparison in machine learning research. Differences in implementation details, hyperparameter tuning procedures, and evaluation protocols can significantly influence reported performance, making it difficult to determine whether performance differences are caused by the model itself or by differences in experimental setup [10, 19, 27].

2.4 Reproducibility and Fair Comparison in Machine Learning

Reproducibility is a fundamental principle of scientific research, as it allows independent researchers to verify results and build upon previous work. In machine learning research, reproducibility has become an increasingly important topic in recent years, as several studies have shown that published results are often difficult to reproduce due to missing experimental details, differences in hyperparameter tuning, and variations in evaluation protocols [19]. As machine learning experiments often involve many design choices, small differences in experimental setup can lead to large differences in reported performance [19].

It is important to distinguish between reproducibility and replicability. Reproducibility refers to obtaining the same results using the same code, data, and experimental setup, while replicability refers to obtaining similar results when independently re-implementing a method based on its description [19]. In machine learning research, replicability is often more challenging, as small implementation differences or undocumented experimental choices can lead to significant performance differences.

Similar concerns have been raised in the field of recommender systems. DaisyRec [27] argues that the field lacks standardized evaluation benchmarks, which often leads to unfair comparisons between models. The authors show that the performance of recommender systems is influenced not only by the model itself, but also by many experimental factors such as data preprocessing, dataset splitting strategies, hyperparameter tuning, and evaluation protocols [27]. They categorize these factors into model-dependent and model-independent factors and argue that fair comparison requires controlling the model-independent factors, as differences in these can lead to large performance differences even when the same model is used [27].

These observations are particularly relevant when comparing different implementations of the same algorithm, where performance differences may be caused by differences in experimental setup rather than differences in the algorithm itself [10, 27, 19]. This thesis therefore revisits the comparison between the official GRU4Rec implementation and the RecPack implementation under a controlled and transparent experimental setup in order to better understand the sources of the reported performance differences.

Chapter 3

Methodology

3.1 GRU4Rec

GRU4Rec is a neural architecture designed for session-based recommendation [12]. It leverages Gated Recurrent Units (GRUs) [6] to model sequential user behavior within a session and predict the next item based on previous interactions. By maintaining a hidden state that is updated at each time step, the model captures the evolving short-term preferences of the user throughout the session.

3.1.1 Session Representation

In GRU4Rec, a session is represented as an ordered sequence of item identifiers:

$$S = (i_1, i_2, \dots, i_t),$$

where each i_k corresponds to an item interacted with during the session. These item identifiers form the input sequence of the model. Before the sequence can be processed by the recurrent layer, each item identifier must be converted into a vector representation.

Two representation strategies are supported by the original GRU4Rec implementation [12, 11]:

- **One-hot representation:** each item is represented by a sparse vector of length equal to the number of items in the catalog. The position corresponding to the current item is set to one, while all other positions are zero. This corresponds to the original no-embedding configuration of GRU4Rec.
- **Embedding representation:** each item identifier is mapped to a learned dense vector of lower dimensionality. The embedding vectors are optimized jointly with the rest of the network during training.

The choice between both approaches determines how item information is represented within the model. One-hot representations encode item identity explicitly, whereas embeddings represent items using learned dense vectors that can capture latent relationships between items.

The original GRU4Rec implementation supports both representation modes and allows embeddings to be disabled entirely by setting the embedding dimension to zero [11]. In this case, items are represented directly through their one-hot encoding.

RecPack originally only supported the embedding-based variant, making it impossible to reproduce certain configurations of the original implementation without modifying the framework.

Previous work has shown that the no-embedding configuration can perform competitively and may even outperform embedding-based variants in some settings [12, 11]. Consequently, support for both representation modes is important when attempting to faithfully reproduce the behavior of the original GRU4Rec implementation.

3.1.2 Architecture

The GRU4Rec model consists of one or more stacked GRU layers [12], optionally followed by fully connected layers. At each time step, the model receives the encoded representation of the current item as input and updates its hidden state accordingly. This hidden state summarizes the sequence of previous interactions and serves as a representation of the current session state.

The output of the final layer is used to compute scores for candidate items, which are then used to produce a ranked list of items representing their likelihood of being the next interaction in the session. Unlike traditional classification tasks, the objective is not to predict a probability distribution over items, but rather to rank items such that relevant items are assigned higher scores than irrelevant ones [12]. A visual overview of the architecture is shown in Figure 3.1.

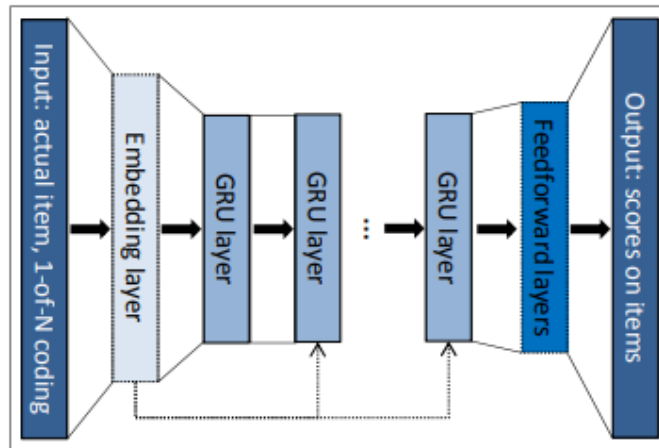


Figure 3.1: GRU4Rec architecture (copied from [12]).

3.1.3 Session-Parallel Mini-Batches

Standard RNN training procedures often rely on a sliding window approach to generate training samples. However, this approach is not well suited for session-based data, as it disrupts the natural sequential structure of sessions. To address this, the authors of GRU4Rec introduced a training strategy based on session-parallel mini-batches [12].

In this approach, each mini-batch processes one interaction per session in parallel. The first mini-batch consists of the first interaction of multiple sessions, the second mini-batch consists of the second interactions, and so on. When a session ends, it is replaced by a new session in the mini-batch, and the corresponding hidden state is reset. This ensures independence between sessions while enabling efficient parallelization on GPUs [12].

A schematic overview of this training procedure is shown in Figure 3.2.

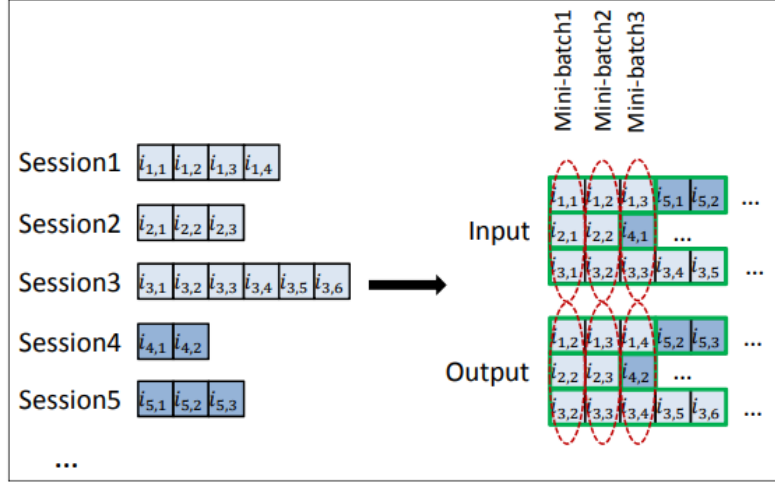


Figure 3.2: Session-parallel mini-batch training (copied from [12]).

3.1.4 Loss Functions

Although GRU4Rec can be trained using pointwise loss functions such as cross-entropy, the session-based recommendation task is inherently a ranking problem rather than a classification problem. The objective is not to predict the exact next item, but to rank the correct item higher than other candidate items. For this reason, ranking-based loss functions have been shown to be more effective [11].

In GRU4Rec, the model is trained to assign a higher score to the true next item in the session (denoted by $\hat{r}_i$) than to sampled negative items (denoted by $\hat{r}_j$). The original formulation supports two main pairwise ranking loss functions:

1. Bayesian Personalized Ranking (BPR) [22]:

The BPR loss encourages the model to rank the positive item higher than negative items by maximizing the difference between their predicted scores:

$$L_{\text{BPR}} = -\frac{1}{N} \sum_j \log \sigma(\hat{r}_i - \hat{r}_j)$$

where $\sigma(\cdot)$ denotes the sigmoid function. This loss penalizes cases where negative items are assigned higher scores than the positive item.

2. TOP1 Loss:

The TOP1 loss, proposed by the authors of GRU4Rec [12], not only encourages correct ranking but also regularizes the scores of negative items:

$$L_{\text{TOP1}} = \frac{1}{N} \sum_j \sigma(\hat{r}_j - \hat{r}_i) + \sigma(\hat{r}_j^2)$$

The first term promotes correct ranking by penalizing negative items that are ranked above the positive item, while the second term pushes the scores of negative items towards zero, which stabilizes training.

Ranking-Max Losses The authors later observed that standard pairwise ranking losses become less effective when many negative samples are used [11]. In such cases, many negative items are already ranked far below the positive item and therefore contribute little to the gradient. As a result, learning can be dominated by uninformative negatives, leading to slower convergence and vanishing gradient effects.

To address this issue, the authors introduced ranking-max loss functions [11], which place greater emphasis on the most competitive negative samples. Instead of treating all negatives equally, these losses apply a softmax weighting over the negative items, assigning higher importance to negatives with higher predicted scores.

The resulting loss functions are defined as:

1. **TOP1-max:**

$$L_{\text{TOP1-max}} = \sum_j s_j [\sigma(\hat{r}_j - \hat{r}_i) + \sigma(\hat{r}_j^2)]$$

2. **BPR-max:**

$$L_{\text{BPR-max}} = -\log \sum_j s_j \sigma(\hat{r}_i - \hat{r}_j)$$

where the weights s_j are defined as a softmax over the negative item scores:

$$s_j = \frac{e^{\hat{r}_j}}{\sum_k e^{\hat{r}_k}}.$$

By focusing the learning process on the most informative negative samples, these loss functions improve training efficiency and ranking performance in practice [11].

The authors further observed that part of the strong performance of TOP1 can be attributed to its score regularization term [11]. Besides encouraging positive items to be ranked above negative items, TOP1 also penalizes large negative-item scores by pushing them towards zero. Experimental results showed that this form of score regularization improves learning, whereas applying conventional ℓ_2 regularization directly to the model parameters tends to degrade performance.

Motivated by these findings, the authors introduced a similar score regularization mechanism into BPR-max. The resulting regularization term penalizes large scores assigned to negative items, with stronger penalties applied to negatives receiving higher softmax weights. The final form of the BPR-max loss is therefore given by:

$$L_{\text{BPR-max}} = -\log \sum_{j=1}^{N_S} s_j \sigma(\hat{r}_i - \hat{r}_j) + \lambda \sum_{j=1}^{N_S} s_j \hat{r}_j^2,$$

where λ controls the strength of the score regularization.

3.1.5 Candidate Sampling During Training

Training recommender systems on datasets with large item catalogs can be computationally expensive, as the model must learn to distinguish the true next item from many alternative items. To make training more efficient, GRU4Rec restricts the set of candidate items considered during loss computation [12, 11].

Rather than evaluating the model against the entire item catalog during training, GRU4Rec uses a candidate sampling strategy based on items that appear in other sessions within the same mini-batch. These items serve as implicit negative examples. Consequently, for a given training instance, the model learns to assign a higher score to the true next item than to the sampled candidate items.

This strategy is used for both the Cross-Entropy and BPR-max objectives. During training, the loss is therefore computed on a subset of candidate items consisting of the positive target item and sampled negatives rather than on the complete item catalog. This substantially reduces computational cost while still providing informative training signals.

It is important to distinguish this training procedure from the evaluation procedure. During validation and prediction, the goal is to generate a ranking over all possible items in order to compute metrics such as Recall@20 and MRR@20. Consequently, both the original GRU4Rec implementation and RecPack perform full-catalog scoring during evaluation.

The main implementation difference therefore occurs during training. The original GRU4Rec implementation performs candidate sampling before score computation, meaning that only sampled items are scored by the output layer. RecPack, on the other hand, computes scores for all items in the catalog and only applies candidate sampling during loss computation. As a result, both implementations optimize similar objectives, but the amount of computation required during training differs substantially. This distinction is discussed in more detail in Chapter 4.

3.1.6 Ranking Behavior

Since GRU4Rec is optimized for ranking rather than classification, the procedure used to compute item rankings plays an important role. Given the predicted scores for candidate items, a rank is assigned to the true next item based on its relative position among all candidates.

Different ranking strategies can be used, particularly in the presence of ties [10, 12] (i.e., when multiple items receive identical scores). The official GRU4Rec implementation supports several tie-handling strategies [10], including standard, conservative, and median ranking, as well as a stochastic tie-breaking approach.

Score ties can arise for various reasons, including limited numerical precision, specific loss configurations, or model behavior that produces similar scores for many items. The authors of GRU4Rec note that a high number of ties may indicate saturation effects or suboptimal training behavior [12]. Regardless of their origin, ties make the choice of ranking strategy particularly important, as different tie-handling approaches can lead to different evaluation outcomes.

3.2 Official GRU4Rec Implementation

The original GRU4Rec implementation was developed in Theano [8, 12] and serves as the reference implementation of the model.

The implementation supports several architectural and training-related features [12, 11, 10]:

- One-hot item representations without embeddings, as well as embedding-based configurations
- Session-parallel mini-batch training
- Candidate sampling from items appearing in other sessions within the same mini-batch during training
- Multiple loss functions, including BPR, TOP1, BPR-max, TOP1-max, and Cross-Entropy

In practice, not all supported loss functions yield equally strong results. Empirical evaluations reported by the authors indicate that BPR-max and Cross-Entropy generally provide the most competitive and stable performance [11], whereas other objectives can be more sensitive to hyperparameter settings.

The implementation further provides flexibility in how item representations are handled. It supports both embedding-based and no-embedding configurations. When embeddings are used, separate embedding matrices can be employed for the input representation and the output scoring layer. Furthermore, the implementation exposes separate dropout parameters for the embedding layer and recurrent hidden states, allowing more fine-grained regularization.

For evaluation, the implementation generates rankings over the complete item catalog in order to compute recommendation metrics such as Recall@20 and MRR@20. In addition, it supports multiple ranking strategies for handling tied scores during evaluation [10]:

- **Standard ranking:** the positive item is ranked above all tied items
- **Conservative ranking:** the positive item is ranked below all tied items
- **Median ranking:** the positive item is assigned the average rank within the tied group
- **Tie-breaking:** small random perturbations are added to the scores to eliminate ties

These evaluation options provide greater control over how tied predictions are handled and can have a noticeable impact on the reported recommendation metrics when large numbers of tied scores occur.

3.3 RecPack Implementation

RecPack is an experimentation toolkit for recommender systems implemented in Python [17]. It provides a PyTorch-based implementation of GRU4Rec [17, 18]. as part of a broader recommendation framework.

Despite following the general architecture of the original model, several differences exist between the official GRU4Rec implementation and the RecPack implementation [10], both in terms of functionality and performance. These differences arise primarily from variations in design choices and training procedures.

The most notable differences include:

- **Loss computation and candidate scoring:** RecPack uses negative sampling during training. However, the model first computes scores for the complete item catalog and subsequently selects the sampled positive and negative item scores used for loss computation. Consequently, the loss is evaluated on sampled items, while the forward pass still performs full-catalog scoring. This differs from the official GRU4Rec implementation, where candidate sampling is integrated into the scoring procedure itself, such that only sampled items are scored. A detailed analysis of this difference is provided in Chapter 4.
- **Mini-Batching Strategy:** In RecPack, mini-batches are processed statically, meaning that sessions are not replaced when they terminate [10]. As a result, computation may be wasted on inactive batch slots. In contrast, the original implementation uses dynamic session replacement to maintain fully utilized batches.
- **Dropout Configuration:** RecPack applies a single shared dropout parameter across both embeddings and hidden layers. This limits flexibility in regularization compared to the official implementation, which uses separate dropout parameters for different components.
- **Ranking Strategy:** RecPack relies on a single default ranking procedure, which corresponds to the standard ranking strategy used in the official GRU4Rec implementation. However, unlike the official implementation, it does not provide support for alternative tie-handling strategies such as conservative or median ranking.

In addition to these differences, further discrepancies exist in aspects such as weight initialization and embedding configurations. A detailed comparison between the claims made in the original comparison paper, the actual behavior of RecPack, and the official GRU4Rec implementation is provided in Table 3.1.

These differences form the basis for the experimental analysis in this thesis, where their impact on model performance is systematically evaluated.

Aspect	Claim in [10]	RecPack	Official GRU4Rec
Candidate scoring during training	Full-catalog scoring reduces efficiency	Computes scores for all items before selecting sampled candidates for the loss	Restricts scoring to sampled candidates before score computation
Mini-batching	Static mini-batching is less efficient than session-parallel batching	Uses sorted, padded mini-batches	Dynamically replaces completed sessions during training
Dropout	Only one shared dropout parameter	Shared dropout configuration	Separate dropout parameters for embeddings and hidden states
Weight initialization	Initialization differs from GRU4Rec	Fixed uniform initialization $[-0.01, 0.01]$	Glorot Uniform initialization
Embeddings	Missing support for certain GRU4Rec configurations	Separate input/output embeddings only	Supports no embeddings, shared embeddings, and separate embeddings

Table 3.1: Summary of implementation differences identified through code inspection and comparison with the claims reported in [10].

Chapter 4

Analysis of Implementation Differences

This chapter presents a detailed comparison between the official GRU4Rec implementation and the RecPack reimplementation. The goal is to identify implementation-level differences that may affect recommendation quality, computational efficiency, and the fairness of experimental comparisons. While both implementations are based on the same underlying architecture, code inspection reveals several important differences in training procedures, model configuration, and evaluation methodology.

A key observation is that several differences occur at different stages of the recommendation pipeline. Therefore, before discussing individual implementation choices, it is important to distinguish between training, validation, and prediction.

4.1 Training, Validation and Prediction

Before discussing the implementation differences, it is important to distinguish between training, validation, and prediction. Several of the differences identified in this chapter only affect one of these stages, and failing to separate them can lead to incorrect conclusions regarding both performance and runtime.

Training During training, the model receives an input item and learns to predict the next item in the session. Depending on the loss function, positive items and negative samples are used to compute gradients and update the model parameters. The candidate scoring strategy discussed later in this chapter primarily affects this stage.

Validation Validation is performed during hyperparameter optimization. Model parameters remain fixed while recommendation metrics are computed on a separate validation set. In this thesis, validation uses the same ranking-based evaluation procedure as the final test evaluation. Consequently, validation requires full-catalog scoring in order to compute metrics such as Recall@20 and MRR@20.

Prediction Prediction refers to the final evaluation phase on the test set. The trained model generates recommendation scores for all candidate items, after which these scores are converted into rankings and evaluated using ranking metrics.

Although the official GRU4Rec implementation provides an optional evaluation mode that ranks the positive item against a restricted candidate set, the default evaluation procedure uses the complete item catalog. The evaluation script explicitly recommends full-catalog evaluation whenever feasible, as restricting the candidate set artificially inflates recommendation metrics.

A crucial distinction is that full-catalog scoring is expected during validation and prediction because a complete ranking over all items is required to compute recommendation metrics. Both the official GRU4Rec implementation and RecPack use full-catalog evaluation in the experiments considered in this thesis. Consequently, the implementation difference discussed in this chapter primarily concerns the training phase rather than the evaluation procedure.

While both implementations perform full-catalog scoring during validation and prediction, RecPack additionally performs full-catalog scoring during training, even when a sampled ranking loss is used. The original GRU4Rec implementation instead restricts score computation to the sampled candidate items during training. This difference has important implications for computational efficiency and scalability.

4.2 Training-Time Differences

4.2.1 Candidate Scoring Strategy

The most important implementation difference concerns how candidate items are scored during training. Although both implementations employ negative sampling for ranking-based objectives, the point at which candidate selection occurs differs substantially.

In the original GRU4Rec implementation, negative sampling is integrated directly into the output layer computation. During training, a candidate set Y is constructed consisting of the positive target item together with a set of sampled negative items:

```
if self.n_sample:
    Y = np.hstack([out_idx, sample])
else:
    Y = out_idx
```

This candidate set is subsequently passed to the model:

```
H_new, Y_pred, sparms, full_params, sidxs = \
    self.model(
        X, self.H, M, R, Y,
        self.dropout_p_hidden,
        self.dropout_p_embed
    )
```

Inside the model, only the selected candidate items are retrieved from the output layer:

```
Sy = self.Wy[Y]
scores = T.dot(hidden, Sy.T) + self.By[Y]
```

Here, Y contains only the positive item and the sampled negatives. Consequently, the output layer computes scores exclusively for this candidate subset. The number

of computed scores is therefore independent of the total catalog size and depends only on the number of sampled candidates [12, 11].

In contrast, RecPack first computes scores for the entire item catalog before selecting the positive and negative samples used by the loss function. This behavior can be observed in the training loop:

```
output, hidden = self.model_(
    true_input_chunk,
    true_hidden
)
```

Inside the model, the output layer is defined as

```
self.lin = nn.Linear(
    hidden_size,
    num_items + 1
)
```

```
out = self.lin(rnn_x)
```

As a result, the tensor `output` contains a prediction score for every item in the catalog. Only afterwards are the positive and negative samples extracted:

```
positive_scores = torch.gather(
    output,
    2,
    targets_chunk.unsqueeze(-1)
).squeeze(-1)

negative_scores = torch.gather(
    output,
    2,
    negatives_chunk
)
```

The loss is therefore computed on sampled items, but the model has already performed a full-catalog forward pass.

This distinction has important computational consequences. Let $|\mathcal{I}|$ denote the catalog size, N the number of sampled candidates, H the hidden dimension, and B the batch size.

For RecPack, the output layer computes

$$B \times H \times |\mathcal{I}|$$

multiplications per prediction step because every item receives a score.

For the original GRU4Rec implementation, only

$$B \times H \times N$$

multiplications are required since only sampled candidates are scored.

Since $N \ll |\mathcal{I}|$, the original implementation performs substantially less computation during training. For example, using 50 negative samples on the Instacart

dataset ($|Z| = 9331$) [13] reduces the size of the output-layer computation by approximately a factor of 180.

This difference only affects training. During evaluation and prediction, the official implementation no longer uses the sampled candidate set. Instead, it computes

```
y = T.dot(y, self.Wy.T) + \
    self.By.flatten()
```

which produces a score for every item in the catalog. The resulting predictions are subsequently returned as

```
preds = np.asarray(
    self.predict(in_idxs)
).T
```

and indexed using the complete item set. Consequently, both implementations perform full-catalog scoring during evaluation and prediction in order to generate rankings over all candidate items. The computational difference described above is therefore specific to the training procedure rather than to evaluation [12].

4.2.2 Batching Strategy

Another important difference concerns mini-batch construction.

The original GRU4Rec implementation employs a dynamic session-parallel mini-batching strategy [12]. Sessions are processed in parallel, and when a session finishes, its batch position is immediately replaced by a new session. This behavior can be seen in the training loop of the official implementation:

```
finished_mask = (end - start <= 1)
n_finished = finished_mask.sum()

iters[finished_mask] = maxiter + np.arange(1, n_finished + 1)
maxiter += n_finished

valid_mask = (iters < len(offset_sessions) - 1)
mask = finished_mask & valid_mask

sessions = session_idx_arr[iters[mask]]
start[mask] = offset_sessions[sessions]
end[mask] = offset_sessions[sessions + 1]
```

Here, `finished_mask` identifies sessions that have reached their final interaction. These finished batch positions are then assigned new session indices through `iters[finished_mask]`. The corresponding `start` and `end` positions are updated to point to the newly inserted sessions. As a result, completed sessions are dynamically replaced during training, keeping the mini-batch as full as possible.

RecPack uses a different strategy. Instead of dynamically replacing finished sessions, it constructs fixed mini-batches of padded sequences. During training, the batch is split into chunks, and rows containing only padding tokens are filtered out:

```
for p, (input_chunk, target_chunk, neg_chunk) in enumerate(
    self._chunk(
        self.bptt,
```



```

        positives_batch ,
        targets_batch ,
        negatives_batch
    )
):
    input_mask = target_chunk != self.pad_token
    true_rows = input_mask.any(axis=1)

    true_input_chunk = input_chunk[true_rows]
    true_target_chunk = target_chunk[true_rows]
    true_neg_chunk = neg_chunk[true_rows]
    true_hidden = hidden[:, true_rows, :]

```

This shows that RecPack does not replace completed sessions with new ones. Instead, it removes rows that no longer contain valid targets for the current chunk. Consequently, the effective batch size can decrease as shorter sessions finish. Although padded rows are masked out before loss computation, the batch still requires additional padding, masking, and slicing operations.

The difference can therefore be summarized as follows: the original GRU4Rec implementation keeps the mini-batch active by inserting new sessions into completed batch slots, whereas RecPack processes a fixed set of padded sessions until the batch is exhausted. This difference is expected to affect computational efficiency, particularly on datasets with highly variable session lengths. RecPack partially mitigates this issue by grouping sessions of similar length into the same mini-batch. As a result, the amount of padding introduced by static batching is reduced. Nevertheless, unlike the original GRU4Rec implementation, completed sessions are not replaced dynamically during training. The practical impact is investigated experimentally in Chapter 7.

4.3 Model Architecture Differences

4.3.1 Embedding Configuration

One of the most important architectural differences concerns how item representations are handled.

The original GRU4Rec implementation supports three configurations [11]: a no-embedding configuration in which one-hot item representations are directly provided to the GRU, a shared embedding configuration, and a configuration with separate embedding matrices for the input representation and output scoring layer.

RecPack originally only supported the latter configuration, requiring separate embedding matrices for the input and output layers [10]. Consequently, several configurations available in the original implementation could not be reproduced directly.

This difference is particularly relevant because the embedding configuration changes both the architecture and the number of trainable parameters. In the no-embedding configuration, item representations are learned implicitly through the recurrent and output layers. When embeddings are introduced, additional parameters are added and the interaction between items and hidden states is modified through the learned embedding space.

As a result, the embedding configuration can influence model capacity, regularization requirements, and optimization behavior. Furthermore, because the original comparison study evaluated RecPack without support for the no-embedding configuration, it is possible that the compared models were not operating under equivalent architectural settings.

The practical impact of this difference is investigated in the ablation study presented in Chapter 7.

4.3.2 Dropout Configuration

The original GRU4Rec implementation exposes separate dropout parameters for embeddings and recurrent hidden states [11]. This allows different levels of regularization to be applied to different parts of the network.

RecPack originally provided only a shared dropout configuration [10]. Consequently, both components were forced to use identical dropout rates.

Although this difference does not fundamentally alter the architecture, it reduces the flexibility of the hyperparameter search space and may prevent the discovery of optimal configurations.

4.3.3 Weight Initialization

The two implementations also differ in their parameter initialization procedures.

RecPack initializes all weight matrices using a fixed uniform distribution

$$W \sim U(-0.01, 0.01),$$

regardless of the dimensions of the layer. In contrast, the original GRU4Rec implementation uses a size-dependent initialization scheme based on the dimensions of the weight matrix:

$$W \sim U\left(0, 2\sqrt{\frac{6}{n_{in} + n_{out}}}\right),$$

where n_{in} and n_{out} denote the number of input and output connections, respectively.

To quantify the practical difference between both initialization schemes, the initialization ranges were computed for several representative hidden dimensions. The resulting ranges for the GRU input weights, recurrent weights, and output weights are shown in Table 4.1.

Hidden Size	Input GRU Weights	Recurrent GRU Weights	Output Weights
64	$U(0, 0.03738)$	$U(0, 0.30619)$	$U(0, 0.03752)$
128	$U(0, 0.03717)$	$U(0, 0.21651)$	$U(0, 0.03744)$
256	$U(0, 0.03676)$	$U(0, 0.15309)$	$U(0, 0.03731)$
512	$U(0, 0.03599)$	$U(0, 0.10825)$	$U(0, 0.03703)$

Table 4.1: Initialization ranges used by the original GRU4Rec implementation for different hidden dimensions. RecPack uses a fixed initialization range of $U(-0.01, 0.01)$ for all weight matrices.

The results demonstrate that the difference is substantial in practice. While the initialization range of the input and output layers remains relatively stable around $0.036 - 0.038$, the recurrent GRU weights are initialized from considerably larger ranges. Depending on the hidden dimension, the upper bound varies between 0.108 and 0.306 , compared to the fixed bound of 0.01 used by RecPack.

Consequently, the recurrent layers of the original GRU4Rec implementation begin training with weights that are between approximately 11 and 31 times larger than those used by RecPack. This difference is unlikely to be purely cosmetic. The motivation behind Glorot-style initialization is to maintain stable activation and gradient magnitudes throughout training by adapting the initialization scale to the dimensions of each layer [9]. In contrast, RecPack applies the same initialization range to all weight matrices regardless of their size. Therefore, the two implementations start optimization from substantially different parameter distributions, which may influence training dynamics, convergence speed, and optimization stability.

4.4 Evaluation Differences

4.4.1 Ranking Under Tied Scores

A final difference concerns the handling of tied prediction scores during evaluation.

The official GRU4Rec implementation supports multiple ranking strategies [10]. Standard ranking places the positive item above all tied items, conservative ranking places it below all tied items, and median ranking assigns the average rank within the tied group.

RecPack originally only supported standard ranking. Consequently, the evaluation behavior differs when large numbers of tied scores occur. While both implementations compute the same ranking metrics, the treatment of tied scores can influence the final ranking assigned to the positive item.

Because ranking metrics such as Recall@K and MRR@K depend directly on item ranks, differences in tie handling may affect the reported evaluation results. The impact of these ranking strategies is investigated experimentally in Chapter 7.

4.5 Computational Considerations

Several of the implementation differences described above are expected to affect computational efficiency.

The candidate scoring strategy directly influences the amount of computation required during training. Because RecPack computes scores for the complete item catalog before selecting the sampled candidates used by the loss function, the computational cost grows with the catalog size. In contrast, the original GRU4Rec implementation restricts score computation to the sampled candidate set.

The batching procedure may also influence computational efficiency. GRU4Rec employs a session-parallel batching strategy that dynamically replaces completed sessions, whereas RecPack processes fixed mini-batches using padded session sequences.

Finally, both implementations perform full-catalog scoring during validation and prediction in order to compute ranking metrics such as Recall@20 and MRR@20.

As a result, evaluation remains computationally expensive regardless of the training procedure.

The extent to which these implementation differences affect runtime is investigated experimentally in Chapter 7.

4.6 Summary

Code inspection revealed several differences between the official GRU4Rec implementation and the RecPack reimplementation. These differences concern candidate scoring during training, batching procedures, embedding support, dropout configuration, weight initialization, and evaluation behavior.

Some of these differences affect the training procedure, while others affect model configuration or evaluation. The following chapters investigate the extent to which these implementation differences contribute to the performance and runtime discrepancies reported in previous comparison studies.

Chapter 5

Experimental Setup

5.1 Datasets and Preprocessing

5.1.1 Datasets Used in the Original Comparison

The original comparison study [10] evaluates GRU4Rec implementations on five publicly available session-based datasets: YooChoose [5], Rees46 [20], Coveo [28], RetailRocket [23], and Diginetica [7]. These datasets originate from real-world e-commerce platforms and have been widely adopted for evaluating session-based recommender systems.

To ensure a faithful reproduction of the original experiments, the same datasets and preprocessing procedures are used in this study. Across all datasets, only interaction events corresponding to item views (e.g., clicks or detail page visits) are retained, as the task is formulated as next-item prediction within sessions.

Sessions are either provided directly or reconstructed using a time-based inactivity threshold, typically set to one hour. Additional preprocessing steps include removing consecutive duplicate items, filtering sessions shorter than two interactions, and discarding infrequent items. Train–test splits follow the time-based protocol defined in the original study [10].

5.1.2 Datasets Used in This Study

To evaluate the behavior of the different GRU4Rec implementations under diverse conditions, several additional publicly available datasets are considered. These datasets vary in domain, scale, and interaction patterns, enabling a comprehensive assessment of model performance across different recommendation scenarios.

The datasets used in this study are summarized below:

- **YooChoose (1/64 split):** The YooChoose dataset [5] consists of e-commerce clickstream data released for the RecSys Challenge 2015. Since session identifiers are provided, no additional sessionization is required. Following common practice, only the most recent $\frac{1}{64}$ of sessions is retained. Validation and test sets are constructed using time-based splits with a one-day validation window and a one-day test window.
- **Last.fm 1K:** The Last.fm 1K dataset contains music listening histories [1]. User interaction streams are segmented into sessions using a one-hour inac-

tivity threshold. The most recent $\frac{1}{32}$ of sessions is retained. Evaluation is performed using time-based splits with seven-day validation and test windows.

- **Instacart:** The Instacart dataset [13] contains online grocery shopping interactions, where individual orders are treated as sessions. The most recent $\frac{1}{64}$ of orders is retained. Validation and test sets are created using seven-day time-based splits.
- **MovieLens:** The MovieLens dataset contains user interactions with movies [2]. Sessions are constructed using a one-day inactivity threshold. The dataset is split into training, validation, and test sets using an 80% / 10% / 10% split based on session order.
- **Taobao:** The Taobao UserBehavior dataset consists of large-scale e-commerce interaction logs [3]. Only page-view events are retained. Sessions are constructed using a one-hour inactivity threshold. The most recent $\frac{1}{256}$ of users is selected. The dataset is split into 80% training, 10% validation, and 10% test sets based on session order.

In addition to the preprocessing steps described above, items that appear in the validation or test sets but not in the training set are removed. This filtering step is necessary because GRU4Rec, as a collaborative filtering model, cannot handle cold-start scenarios for items. Consequently, evaluation is restricted to items observed during training [10, 12].

The use of multiple datasets with diverse characteristics ensures that both small-scale and large-scale recommendation scenarios are represented. This is particularly important when comparing implementations, as certain design choices may have varying impact depending on dataset size and structure. An overview of the dataset statistics after preprocessing is provided in Table 5.1.

Dataset	Split	#Events	#Sessions	#Days	Events/session	#Items
Instacart	Train	230,763	20,406	9	11.31	9,331
	Val	116,230	10,249	7	11.34	
	Test	28,273	2,773	7	10.20	
Last.fm 1K	Train	244,972	14,075	54	17.40	23,613
	Val	13,031	646	7	20.17	
	Test	7,003	388	7	18.05	
MovieLens	Train	67,869	2,495	6848	27.20	4,848
	Val	12,061	311	698	38.78	
	Test	12,744	311	659	40.98	
Taobao	Train	118,828	19,771	1066	6.01	16,988
	Val	15,477	2,390	1	6.48	
	Test	16,714	2,421	6898	6.90	
YooChoose	Train	250,322	59,544	2	4.20	7,123
	Val	47,739	10,473	1	4.56	
	Test	58,469	12,930	1	4.52	

Table 5.1: Dataset statistics after preprocessing

5.2 Evaluation Protocol

All experiments follow the next-item prediction protocol introduced in the GRU4Rec comparison study [10]. Given a session $(i_1, i_2, \dots, i_T)$, evaluation is performed iteratively: at each timestep t , the prefix $(i_1, \dots, i_t)$ is used to predict the next item i_{t+1} . Performance is computed over all valid prefix-target pairs and averaged across the dataset.

Items that do not appear in the training set are excluded from evaluation, as GRU4Rec cannot handle item cold-start scenarios [10, 12].

The models are evaluated using the standard top- K ranking metrics **Recall@20** and **MRR@20**:

- **Recall@20** measures whether the true next item appears within the top-20 ranked items:

$$\text{Recall@20} = \frac{1}{|Q|} \sum_{q \in Q} I(\text{rank}_q \leq 20),$$

where Q denotes the set of evaluation instances.

- **MRR@20** (Mean Reciprocal Rank) measures the average reciprocal rank of the true item, considering only the top-20 positions:

$$\text{MRR@20} = \frac{1}{|Q|} \sum_{q \in Q} \begin{cases} \frac{1}{\text{rank}_q}, & \text{if } \text{rank}_q \leq 20 \\ 0, & \text{otherwise} \end{cases}$$

Recall@20 captures whether the correct item is retrieved, while MRR@20 additionally accounts for how highly it is ranked.

5.3 Hyperparameter Tuning

Hyperparameter optimization was performed independently for each implementation and objective function. For each configuration, 50 trials are performed, with each model trained for 5 epochs during tuning.

The search space used during hyperparameter optimization is summarized in Table 5.2. Optimal hyperparameter configurations for all datasets are reported in Appendix A.

5.4 Training Setup

All hyperparameters are selected based on validation performance using the procedure described above. The best-performing configuration, measured by Recall@20, is retrained on the combined training and validation data for 5 epochs and subsequently evaluated on the test set.

All experiments are conducted on a system equipped with an NVIDIA RTX 3060 mobile GPU.

Table 5.2: Hyperparameter search space used for models.

Parameter	Search Range
Hidden size	64 – 512 (step 32)
Batch size	32 – 256 (step 16)
Learning rate	0.01 – 0.25 (step 0.005)
Embedding dropout	0.0 – 0.5 (step 0.05)
Hidden dropout	0.0 – 0.7 (step 0.05)
Momentum	0.0 – 0.9 (step 0.05)
Number of components	{0, 64, 128, 256, 512}
Optimizer	{Adagrad, SGD}
BPR-max regularization (<i>bpreg</i>)	0.0 – 2.0 (step 0.05)
Final activation	{ELU-0.5, ELU-1, Linear}

Chapter 6

Experiments

This chapter describes the experiments conducted to evaluate the differences between the official GRU4Rec implementation and the RecPack reimplementations. The experiments are designed to investigate whether the performance differences reported in the original comparison study can be reproduced, to assess the impact of specific implementation choices, and to determine how these differences affect recommendation quality and computational efficiency.

The experimental evaluation consists of four stages. First, the original comparison study is reproduced in order to establish a baseline and verify the reported findings. Second, a series of ablation studies investigate the impact of individual implementation differences identified in Chapter 4. Third, runtime experiments analyze the computational implications of these implementation choices. Finally, both implementations are evaluated on a diverse collection of datasets using a controlled and consistent experimental setup.

6.1 Reproduction of the Original Comparison

The first experiment reproduces the comparison study of [10]. The goal is to determine whether the reported performance differences between RecPack and the official GRU4Rec implementation can be reproduced using the publicly available source code and the experimental protocol described by the authors.

Three RecPack configurations are evaluated:

- **Out-of-the-box:** the original RecPack implementation.
- **Minor fixes:** replacement of the original weight initialization scheme and removal of hardcoded hyperparameters.
- **Major fixes:** introduction of separate dropout parameters for embeddings and hidden states.

The reproduced results are compared with the official GRU4Rec implementation in order to assess whether the modifications proposed by the original comparison study explain the reported performance gap.

6.2 Ablation Studies

The second stage consists of a series of controlled ablation experiments that investigate individual implementation differences identified in Chapter 4. Each modification is evaluated independently in order to isolate its effect on recommendation quality.

The following implementation aspects are considered:

- **Embedding configuration:** comparison of embedding-based and no-embedding models.
- **Ranking strategy:** analysis of the effect of different tie-handling procedures during evaluation.
- **Dropout parameterization:** comparison of shared and decoupled dropout configurations.

The objective of these experiments is to determine which implementation differences have a meaningful impact on recommendation quality and therefore need to be considered when comparing implementations.

6.3 Runtime Analysis

In addition to recommendation quality, several experiments investigate computational efficiency.

The first runtime experiment measures training and evaluation time separately for both implementations. This allows the contribution of validation and evaluation to the overall runtime to be quantified and determines whether runtime differences primarily originate from the training procedure.

The second runtime experiment investigates the effect of RecPack’s static batching strategy. Additional instrumentation is added to the training loop in order to measure chunk occupancy, padding-related statistics, and the amount of time spent on masking and filtering operations. These measurements provide insight into the practical consequences of the batching differences discussed in Chapter 4.

6.4 Evaluation on Additional Datasets

The final experiment evaluates both implementations on the datasets described in Chapter 5. In contrast to the original comparison study, all models are tuned independently using the same hyperparameter optimization procedure.

Furthermore, the modified RecPack implementation incorporates the findings of the ablation study, including support for disabling embeddings, separate dropout parameters, and improved evaluation procedures.

The objective of this experiment is to determine whether the performance differences reported in the original comparison persist after controlling for the implementation differences identified in Chapter 4. Evaluating both implementations across multiple datasets also makes it possible to assess how sensitive the observed differences are to dataset characteristics such as scale, sparsity, session length, and domain.

Together, these experiments provide a comprehensive assessment of both recommendation quality and computational efficiency, allowing the conclusions of the original comparison study to be re-examined under a more controlled experimental setup.

Chapter 7

Results

7.1 Reproduction of the Original Experiments

The authors of the comparison study evaluate RecPack on several publicly available datasets (introduced in the previous chapter) and analyze the impact of different implementation adjustments. In their experiments, three variants of RecPack are considered [10]:

- **Out-of-the-box:** The original RecPack implementation, with minor adjustments to enable GPU-based evaluation.
- **Minor fixes:** Replacement of the weight initialization scheme with Glorot uniform and removal of hardcoded hyperparameters (e.g., `bpreg`).
- **Major fixes:** Introduction of separate dropout parameters for embeddings and hidden states.

Based on these experiments, the authors report that missing features can lead to a performance degradation of up to 23%, which increases to approximately 51% when additional implementation issues are considered [10]. Their source code is publicly available¹.

To verify these claims, the experiments were reproduced on the Coveo dataset under comparable conditions. All models were trained for 5 epochs using the corresponding subset of hyperparameters reported as optimal for this dataset. The results are summarized in Table 7.1.

Due to computational constraints, the reproduction of the original experiments is primarily conducted on the Coveo dataset. This dataset is representative in terms of scale and complexity and is sufficient to highlight the discrepancies between the reported and reproduced results.

To verify that the observed behavior is not dataset-specific, additional experiments are conducted on a subset of other datasets in later sections.

Contrary to the findings reported in the original study, the reproduced results indicate that the applied modifications do not improve RecPack’s performance. In fact, both the minor and major fixes lead to a slight decrease in performance, particularly for the BPR-max objective. This is unexpected, as these modifications are intended to align RecPack more closely with the original GRU4Rec implementation.

¹https://github.com/hidasib/gru4rec_third_party_comparison

Table 7.1: Effect of implementation modifications on RecPack performance

Configuration	BPR-max		Cross-Entropy	
	Recall@20	MRR@20	Recall@20	MRR@20
Out-of-the-box	0.1855	0.0541	0.2251	0.0709
Minor fixes (init)	0.1837	0.0541	0.2220	0.0690
Major fixes (dropout)	0.1512	0.0433	0.2193	0.0685
Official GRU4Rec	0.3101	0.0989	0.2917	0.0943

This discrepancy suggests that the experimental setup used in the original comparison may introduce unintended biases. Several potential issues can be identified. First, the study does not evaluate RecPack without embeddings, even though the original GRU4Rec work consistently reports that disabling embeddings (i.e., using one-hot representations) yields the best performance [12]. As RecPack enforces the use of embeddings by default, this design choice may disadvantage its performance relative to the official implementation.

Second, the authors reuse hyperparameter configurations that were optimized for the official GRU4Rec implementation when evaluating RecPack [10]. Given the known architectural and training differences between the two implementations, such configurations are unlikely to be optimal for RecPack. This may further bias the comparison in favor of the official implementation.

Overall, the reproduction results indicate that the reported performance gap cannot be solely attributed to the modifications considered in the original study. Instead, they highlight the importance of controlled experimental design and independent hyperparameter optimization when comparing different implementations of the same model.

7.2 Ablation Study

This section investigates several implementation differences between RecPack and the original GRU4Rec implementation through controlled experiments and targeted analysis. In particular, the study evaluates the effect of the embedding configuration, ranking strategy, and dropout parameterization. The goal is to determine whether these implementation choices materially affect recommendation quality and whether they should be incorporated into the RecPack implementation.

7.2.1 Effect of Removing Embeddings

To assess the impact of the embedding configuration, the reproduction experiment was repeated using a modified version of RecPack that supports disabling embeddings (i.e., using one-hot item representations). The results are summarized in Table 7.2.

The results show that removing embeddings consistently improves performance for all RecPack variants when using the BPR-max objective. The improvements are substantial, especially for the configuration with decoupled dropout parameters, where both Recall@20 and MRR@20 increase considerably.

Table 7.2: Effect of disabling embeddings across RecPack modifications on the Coveo dataset.

Objective / Variant	With Embeddings		No Embeddings		Change	
	Recall@20	MRR@20	Recall@20	MRR@20	Δ Recall	Δ MRR
BPR-max / Out-of-the-box	0.1855	0.0541	0.2117	0.0591	+0.0262	+0.0050
BPR-max / Minor fixes	0.1837	0.0541	0.2117	0.0591	+0.0279	+0.0050
BPR-max / Major fixes	0.1512	0.0433	0.2035	0.0568	+0.0523	+0.0135
CE / Out-of-the-box	0.2251	0.0709	0.2249	0.0715	-0.0002	+0.0006
CE / Minor fixes	0.2220	0.0690	0.2246	0.0713	+0.0025	+0.0024
CE / Major fixes	0.2193	0.0685	0.2224	0.0697	+0.0030	+0.0011

For the Cross-Entropy objective, the effect is much smaller. Performance remains largely unchanged, with only marginal differences in Recall@20 and MRR@20. This indicates that the impact of the embedding configuration is strongly dependent on the optimization objective.

A likely explanation is the increased model complexity introduced by embeddings. In the default RecPack implementation, separate input and output embeddings are always learned, which substantially increases the number of trainable parameters. Removing embeddings produces a more compact model that is easier to optimize and less prone to unstable representations.

This effect is particularly important for BPR-max. As a pairwise ranking objective, BPR-max directly depends on the relative score differences between positive and negative items and is therefore highly sensitive to noisy or unstable item representations. Embedding-based representations can introduce additional variance into these score differences, leading to degraded ranking quality. By contrast, Cross-Entropy relies on a softmax-based objective over the entire item set, producing smoother gradients and greater robustness to such fluctuations.

Overall, the results demonstrate that support for a no-embedding configuration is a significant improvement for RecPack, particularly for ranking-based objectives such as BPR-max. Since the original GRU4Rec implementation consistently reports optimal configurations with embedding size equal to zero [12], this functionality should be considered an essential feature for achieving a fair comparison.

7.2.2 Effect of Ranking Strategy

During hyperparameter tuning, several configurations produced extremely large numbers of tied prediction scores. This has important implications for evaluation, as the selected ranking strategy determines how ties are resolved.

Under the standard ranking strategy, the positive item is ranked above all items with identical scores. Consequently, configurations that produce many ties can obtain artificially inflated Recall@K and MRR@K values, despite having poorly calibrated predictions. This can bias hyperparameter tuning towards unstable models that appear highly performant only because of optimistic tie handling.

In contrast, conservative ranking places the positive item below all tied items, while median ranking assigns the average rank among tied items. Both approaches provide a more robust estimate of model quality in the presence of large numbers of ties.

To illustrate this effect, the same trained model was evaluated using different ranking strategies. The results are summarized in Table 7.3.

Table 7.3: Impact of ranking strategy under extreme tie conditions on the Instacart dataset.

Ranking	Recall@20	MRR@20	Avg Std	Avg Ties	Tie Rate
Standard	0.9504	0.9418			
Conservative	0.0009	0.0004	0.000015	9193.60	99.85%
Median	0.0009	0.0004			

The results clearly show that evaluation metrics can vary drastically despite identical model outputs. The extremely low score variance and near-perfect tie rate indicate that the model is effectively assigning almost identical scores to nearly all items. Under standard ranking, this behavior results in near-perfect Recall@20 and MRR@20 scores, whereas conservative and median ranking reveal that the model is essentially unable to distinguish between items.

These findings demonstrate that ranking strategy is not merely an implementation detail, but a critical component of reliable evaluation. Standard ranking can heavily favor degenerate models during hyperparameter tuning, potentially leading to the selection of suboptimal configurations. Incorporating more robust tie-handling strategies, such as conservative or median ranking, is therefore essential for fair and meaningful comparisons and should be considered in future versions of RecPack.

7.2.3 Impact of Decoupled Dropout

To assess the impact of decoupled dropout parameters during training, two separate hyperparameter searches were conducted: one using a shared dropout configuration, where the same dropout rate is applied to both embeddings and hidden states, and one using decoupled dropout parameters, where both components are tuned independently. The results are summarized in Table 7.4.

Table 7.4: Effect of decoupling dropout parameters on the Instacart dataset.

Objective	Config	Recall@20	MRR@20	p_{embed}	p_{hidden}	ΔRecall	ΔMRR
BPR-max	Shared	0.1548	0.0453	0.55	0.55	–	–
	Decoupled	0.1576	0.0483	0.40	0.25	+0.0027	+0.0030
Cross-Entropy	Shared	0.1760	0.0581	0.65	0.65	–	–
	Decoupled	0.1748	0.0570	0.50	0.70	-0.0011	-0.0010

The results indicate that decoupled dropout parameters have only a limited impact on the final recommendation performance. For the BPR-max objective, a small improvement is observed when tuning embedding and hidden-state dropout independently. In contrast, the Cross-Entropy objective slightly favors the shared configuration.

Although the observed differences are relatively small, decoupled dropout still provides additional flexibility during hyperparameter optimization. The original GRU4Rec implementation consistently exposes separate dropout parameters for embeddings and hidden states, and the best-performing configurations reported in the

literature frequently use different values for both components. This suggests that the optimal level of regularization may differ between the embedding layer and the recurrent hidden state.

Therefore, even if the direct performance gains are limited in this experiment, supporting decoupled dropout remains a meaningful improvement for RecPack, as it increases configurability and enables a closer alignment with the original GRU4Rec implementation.

7.3 Runtime Analysis

7.3.1 Training versus Evaluation Runtime

To better understand where runtime is spent, training and evaluation times were measured separately for both implementations on the LastFM1K dataset. This allows us to determine whether evaluation constitutes a substantial portion of the overall runtime.

Table 7.5 reports the runtime breakdown for both implementations using their respective best-performing hyperparameter configurations.

Implementation	Training (s)	Evaluation (s)	Evaluation Share
Official GRU4Rec	27.31	0.98	3.47%
RecPack	2055.35	4.80	0.23%

Table 7.5: Training and evaluation runtime on LastFM1K.

The results show that validation and evaluation account for only a small fraction of the total runtime for both implementations. For the official GRU4Rec implementation, evaluation represents 3.47% of the total runtime, while for RecPack it represents only 0.23%.

These findings indicate that validation and evaluation are not major contributors to runtime. Instead, the overwhelming majority of runtime is spent during model training. Combined with the profiling results presented in the following section, this suggests that the runtime discrepancy between both implementations primarily originates from differences in their training procedures.

7.3.2 Impact of Static Batching

The analysis in Chapter 4 identified an important difference in mini-batch construction between the official GRU4Rec implementation and RecPack. Whereas GRU4Rec employs a session-parallel mini-batching strategy that continuously replaces completed sessions during training, RecPack constructs fixed mini-batches of padded session sequences [10, 12].

To assess the practical impact of this design choice, additional instrumentation was added to the RecPack training loop. Two metrics were recorded. The first is the *active-row ratio*, defined as the proportion of rows within a training chunk that contain at least one valid target item after padding has been applied. Rows consisting entirely of padding tokens are considered inactive. The reported values correspond to the average active-row ratio across all chunks processed during training. The

second metric is the *batching runtime share*, which measures the proportion of total epoch runtime spent on masking, filtering, tensor slicing, and other operations required to construct valid mini-batches.

The results are presented in Table 7.6.

Table 7.6: Impact of static batching on training efficiency.

Dataset	Active Rows	Padding	Length Spread	Batching Time (s)	Runtime Share
YooChoose	8.2%	91.8%	36.84	9.32	21.9%
Instacart	25.1%	74.9%	38.97	3.55	15.9%
LastFM1K	6.7%	93.3%	243.78	23.05	34.8%

At first glance, the active-row ratios appear surprisingly low. However, these values should not be interpreted as direct computational waste. The metric measures the average occupancy of training chunks after padding and chunking, rather than the proportion of computations performed by the neural network. Before each forward pass, RecPack identifies rows containing only padding tokens and removes them from the tensors passed to the GRU. Consequently, the recurrent network itself processes only active session positions.

Nevertheless, static batching introduces additional work outside the neural network. For every training chunk, RecPack must construct masks, identify active rows, create filtered tensors, and update the corresponding hidden states. These operations incur a measurable computational cost. Depending on the dataset, batching-related operations account for between 15.9% and 34.8% of the total epoch runtime. In absolute terms, this corresponds to between 3.55 and 23.05 seconds per epoch.

The largest overhead is observed on LastFM1K, where batching operations account for more than one third of the total epoch runtime. This dataset also exhibits substantially greater variation in session lengths than YooChoose and Instacart. As a result, a larger number of partially padded chunks must be processed, increasing the amount of masking and filtering required during training.

Combined with the findings of the previous section, which showed that validation and evaluation account for only a small fraction of total runtime, these results indicate that the observed runtime differences are primarily associated with the training procedure itself. While RecPack successfully avoids performing GRU computations on padded positions, managing padded session sequences still introduces a non-negligible overhead. This overhead becomes increasingly pronounced as session length variability increases, suggesting that the session-parallel batching strategy employed by the original GRU4Rec implementation provides a more computationally efficient approach for datasets with highly variable session lengths.

7.4 Evaluation on Additional Datasets

Finally, both implementations are evaluated on the additional datasets introduced in Chapter 5. Unlike the original comparison study, all models are independently tuned using the same hyperparameter optimization procedure. Furthermore, the modified RecPack implementation incorporates the findings of the previous ablation studies, including support for disabling embeddings, decoupled dropout parameters, and conservative ranking during evaluation.

The objective of this experiment is to determine whether the performance differences reported in the original comparison persist after controlling for the implementation differences identified in Chapter 4. Evaluating both implementations across multiple domains additionally makes it possible to assess how sensitive these differences are to dataset characteristics such as scale, sparsity, and session structure.

7.4.1 YooChoose

Table 7.7: YooChoose (1/64 split): RecPack vs Official GRU4Rec under next-item prediction.

Model	Recall@20	MRR@20
Official GRU4Rec (Cross-Entropy)	0.6657	0.2883
Official GRU4Rec (BPR-max)	0.6694	0.2864
RecPack GRU4Rec (Cross-Entropy)	0.6480	0.2784
RecPack GRU4Rec (BPR-max)	0.6327	0.2725

The results on YooChoose show that the modified RecPack implementation performs relatively close to the official implementation for both objectives. The Recall@20 differences remain below four percentage points, while the MRR@20 values are highly similar. This suggests that, on this dataset, the modifications introduced in RecPack are sufficient to reproduce most of the behavior of the original GRU4Rec model.

7.4.2 LastFM1K

Table 7.8: LastFM1K: RecPack vs Official GRU4Rec under next-item prediction.

Model	Recall@20	MRR@20
Official GRU4Rec (Cross-Entropy)	0.4425	0.2959
Official GRU4Rec (BPR-max)	0.4629	0.2789
RecPack GRU4Rec (Cross-Entropy)	0.3989	0.2944
RecPack GRU4Rec (BPR-max)	0.4027	0.2749

On LastFM1K, the official implementation consistently outperforms RecPack. The difference is particularly visible in Recall@20, whereas MRR@20 remains relatively similar. This indicates that both implementations often rank relevant items similarly once they appear in the recommendation list, but that the official implementation retrieves the correct item more frequently overall. Compared to YooChoose, the larger performance gap suggests that implementation differences become more important on datasets with longer and more complex sequential patterns.

7.4.3 Instacart

For Instacart, both implementations achieve very similar performance when optimized using the Cross-Entropy objective. The gap becomes larger for BPR-max,

Table 7.9: Instacart: RecPack vs Official GRU4Rec under next-item prediction.

Model	Recall@20	MRR@20
Official GRU4Rec (Cross-Entropy)	0.1800	0.0577
Official GRU4Rec (BPR-max)	0.1856	0.0608
RecPack GRU4Rec (Cross-Entropy)	0.1705	0.0563
RecPack GRU4Rec (BPR-max)	0.1627	0.0493

where the official implementation obtains noticeably higher Recall@20 and MRR@20 values. This observation is consistent with the ablation experiments, which indicated that BPR-max is generally more sensitive to implementation-specific details than Cross-Entropy.

7.4.4 MovieLens

Table 7.10: MovieLens: RecPack vs Official GRU4Rec under next-item prediction.

Model	Recall@20	MRR@20
RecPack GRU4Rec (Cross-Entropy)	0.1060	0.0305
RecPack GRU4Rec (BPR-max)	0.0867	0.0253
Official GRU4Rec (Cross-Entropy)	0.1078	0.0244
Official GRU4Rec (BPR-max)	0.1165	0.0323

The Cross-Entropy results on MovieLens are nearly identical, with RecPack even achieving a slightly higher MRR@20. In contrast, the official implementation again obtains substantially stronger results when using BPR-max. Together with the previous datasets, these findings suggest that the remaining performance differences between both implementations are considerably larger for pairwise ranking objectives than for Cross-Entropy optimization.

7.4.5 Taobao

Table 7.11: Taobao: RecPack vs Official GRU4Rec under next-item prediction.

Model	Recall@20	MRR@20
RecPack GRU4Rec (Cross-Entropy)	0.1112	0.0521
RecPack GRU4Rec (BPR-max)	0.1109	0.0504
Official GRU4Rec (Cross-Entropy)	0.2128	0.1074
Official GRU4Rec (BPR-max)	0.2063	0.0983

The largest performance gap is observed on Taobao. For both optimization objectives, the official implementation achieves approximately twice the Recall@20 and MRR@20 values obtained by RecPack. Unlike the previous datasets, the gap remains substantial even for Cross-Entropy.

One possible explanation is that the scale and sparsity of Taobao amplify implementation-level differences. The dataset combines a large item catalog with relatively short

sessions, making the prediction task substantially more challenging. Under these conditions, differences in candidate scoring, batching, and optimization may have a greater impact on the quality of the learned representations.

Overall, the results indicate that the modified RecPack implementation is capable of reproducing the behavior of the official implementation reasonably well on several datasets, particularly when using the Cross-Entropy objective. However, substantial differences remain for BPR-max and on the large-scale Taobao dataset, suggesting that implementation details continue to influence recommendation quality even after aligning the most important architectural and evaluation choices.

7.5 Summary of Findings

The experiments presented in this chapter provide several important insights into the differences between the official GRU4Rec implementation and the RecPack reimplementation.

First, reproducing the original comparison study revealed that several of the reported implementation differences were either overstated or incorrectly characterized. In particular, the original study attributed a substantial portion of the performance gap to implementation choices [10] that, upon closer inspection, either behaved differently than described or had a smaller impact than originally suggested.

Second, the ablation experiments demonstrated that not all implementation differences contribute equally to recommendation quality. The introduction of separate dropout parameters and support for disabling embeddings reduced part of the performance gap between both implementations. The ranking strategy used during evaluation was also found to have a significant influence on the reported results, particularly in the presence of tied prediction scores. In contrast, some differences, such as weight initialization, primarily affect optimization behavior rather than the underlying model architecture.

Third, the runtime analysis showed that validation and evaluation account for only a small fraction of total runtime. Instead, the majority of computational cost originates from the training procedure itself. Additional profiling of RecPack’s batching mechanism revealed that managing padded session sequences introduces a measurable overhead, particularly on datasets with highly variable session lengths. These findings suggest that the runtime differences between both implementations are primarily caused by differences in candidate scoring and batching strategies rather than by evaluation.

Finally, the evaluation on additional datasets demonstrated that the performance gap between both implementations is highly dataset-dependent. Unlike the original comparison study, all models were independently tuned using the same hyperparameter optimization procedure, ensuring that neither implementation was disadvantaged by suboptimal parameter choices. On YooChoose, Instacart, and MovieLens, the modified RecPack implementation achieved performance relatively close to the official implementation, particularly when using the Cross-Entropy objective. Larger differences remained for BPR-max, indicating that pairwise ranking objectives are more sensitive to implementation details. The largest discrepancy was observed on Taobao, where the official implementation substantially outperformed RecPack under both optimization objectives.

Overall, the results indicate that several of the conclusions drawn in the original

comparison study should be interpreted with caution. While implementation details clearly influence both recommendation quality and computational efficiency, their impact is considerably more nuanced than originally suggested. Furthermore, the experiments demonstrate the importance of independently tuning both implementations before drawing conclusions regarding relative performance. After aligning the most important architectural and evaluation choices and performing comparable hyperparameter optimization, the performance gap between both implementations becomes substantially smaller on most datasets, although notable differences remain in large-scale and sparse recommendation settings.

Chapter 8

Discussion

The experiments demonstrate that multiple implementation-level differences substantially influence both the efficiency and evaluation behavior of GRU4Rec-based recommender systems. Although RecPack and the original GRU4Rec implementation nominally optimize the same objective, differences in embedding configuration, ranking behavior, batching strategy, and scoring procedures lead to meaningful discrepancies in both performance and runtime.

8.1 Revisiting the Original Comparison

The results of this work suggest that the conclusions of the original comparison study should be interpreted with caution. Although the study correctly identified several implementation differences between RecPack and the official GRU4Rec implementation, the practical impact of these differences appears to be more nuanced than originally reported.

A central issue concerns the experimental setup used for the comparison. The original study reused hyperparameters optimized for the official GRU4Rec implementation rather than independently tuning the RecPack implementation [10]. This implicitly assumes that both implementations share similar optimization behavior and architectural characteristics. However, the analysis presented in this work demonstrates that this assumption does not necessarily hold. Differences in embedding support, dropout configuration, ranking behavior, and training procedures alter the effective hypothesis space of the model and can therefore influence the optimal hyperparameter configuration.

The ablation experiments further demonstrate that several unsupported or differently implemented features have a measurable impact on recommendation quality. In particular, support for disabling embeddings and the choice of ranking strategy during evaluation significantly affected the reported results. Consequently, part of the performance gap reported in the original comparison appears to be attributable to differences in configuration space and evaluation methodology rather than to fundamental shortcomings of either implementation.

These findings highlight the importance of independently tuning competing implementations and carefully aligning evaluation procedures before drawing conclusions regarding relative model quality. Comparing implementations solely on the basis of a shared high-level architecture can be misleading when important implementation details differ.

8.2 Why Does the Gap Persist for BPR-max?

One of the most consistent findings throughout the experiments is that the performance gap between RecPack and the official implementation remains considerably larger for BPR-max than for the Cross-Entropy objective.

After introducing support for missing features and independently tuning both implementations, the Cross-Entropy results become relatively similar across most datasets. In contrast, the official implementation continues to outperform RecPack under BPR-max, particularly on larger datasets.

A possible explanation is that BPR-max is inherently more sensitive to implementation details than Cross-Entropy. Whereas Cross-Entropy directly optimizes the likelihood of the next item, BPR-max relies on pairwise comparisons between positive and negative samples [11]. As a result, the effectiveness of the objective depends strongly on how candidate items are sampled and scored during training.

The official GRU4Rec implementation was specifically designed around sampled ranking losses and computes scores only for the sampled candidates during training [12]. RecPack, however, performs full-catalog scoring during training before selecting the positive and negative items used by the loss. Although both approaches optimize mathematically similar objectives, the resulting optimization dynamics differ substantially.

The persistence of the performance gap under BPR-max therefore suggests that some implementation differences continue to influence the training process even after aligning the most visible architectural and evaluation choices. While the experiments identify several important contributing factors, fully isolating the remaining causes would require a more detailed investigation of the optimization behavior of both implementations.

8.3 Generalization Across Datasets

The experiments reveal a clear pattern. After aligning important implementation details and independently tuning both implementations, the remaining performance differences are relatively small on several datasets, particularly for the Cross-Entropy objective. However, substantial differences remain on larger and sparser datasets, most notably Taobao.

This suggests that implementation-level choices become increasingly important as dataset scale grows. While such differences may have limited practical impact on smaller benchmarks, they can significantly influence both recommendation quality and computational efficiency in large-scale recommendation scenarios.

8.4 Implications for Reproducibility

The experiments conducted in this work demonstrate that relatively small implementation-level differences can substantially influence both the reported recommendation quality and the computational behavior of recommender systems. Differences in embedding configuration, batching strategy, ranking procedure, and scoring implementation were all shown to affect either the final evaluation metrics, the optimization process, or the runtime characteristics of the models.

These findings highlight the importance of careful implementation alignment when comparing recommender system architectures. Even when two implementations nominally optimize the same objective function, differences in supported configurations and internal training behavior may lead to substantially different outcomes. As a result, direct comparison between implementations becomes difficult unless these differences are explicitly controlled.

The experiments further demonstrate that hyperparameter optimization plays a critical role in fair evaluation. Reusing hyperparameters optimized for one implementation on another implicitly assumes that both systems share identical optimization behavior and architectural characteristics, which is often not the case. Consequently, independent hyperparameter tuning is necessary to ensure meaningful and reproducible comparisons.

Overall, this work emphasizes that reproducibility in recommender systems extends beyond reproducing the same high-level architecture or loss function [27, 19]. Implementation details, evaluation methodology, and optimization procedures all contribute significantly to the final reported performance and should therefore be documented and controlled carefully in future comparative studies.

8.5 Research Questions Revisited

RQ1: To what extent are the reported performance differences between the official GRU4Rec implementation and the RecPack implementation caused by implementation differences rather than differences in experimental setup and hyperparameter tuning?

The experiments demonstrate that implementation-level differences have a substantial impact on both recommendation performance and runtime efficiency. Features such as embedding configuration, ranking behavior, batching strategy, and scoring implementation significantly affect the reported results. In addition, independent hyperparameter optimization considerably reduced the performance gap between both implementations, indicating that part of the originally reported discrepancy was caused by differences in tuning procedure rather than by the recommendation architecture itself.

RQ2: Does the experimental setup used in the original comparison lead to a fair comparison between the official GRU4Rec implementation and RecPack?

The findings suggest that the original comparison was not fully fair. The RecPack implementation did not support several important configurations available in the original GRU4Rec implementation, while the same hyperparameter settings were reused across implementations despite architectural differences. Furthermore, ranking behavior under tied scores could artificially inflate evaluation metrics. These factors likely contributed to an overestimation of the performance gap between both implementations.

RQ3: How large is the performance difference between the official GRU4Rec implementation and RecPack across different datasets, and is this difference consistent across datasets?

After aligning important implementation details and performing independent hyperparameter optimization, the performance gap between both implementations

became substantially smaller on several datasets. However, the difference was not fully consistent across datasets. On small and medium-scale datasets, RecPack achieved competitive recommendation quality, whereas larger-scale datasets exposed remaining efficiency and scalability limitations. This indicates that the observed differences depend strongly on dataset characteristics and implementation choices.

8.6 Limitations

While this work provides a detailed comparison between RecPack and the official GRU4Rec implementation, several limitations should be acknowledged.

First, the analysis primarily focuses on the RecPack implementation and its comparison with the original GRU4Rec implementation. The original comparison study evaluated multiple third-party implementations, which were not included in this work. Consequently, additional research is required to determine whether the observed discrepancies also generalize to other GRU4Rec reimplementations.

Second, only a limited number of datasets were used throughout the experiments. Although the selected datasets cover multiple domains and vary considerably in scale and interaction behavior, future work could extend the evaluation to additional recommendation benchmarks in order to further validate the findings.

Another limitation concerns the hyperparameter optimization procedure. Due to hardware and runtime constraints, the search space was limited to 50 hyperparameter configurations per experiment. Furthermore, several datasets had to be substantially subsampled in order to keep experimentation computationally feasible. As a result, it is possible that certain implementations or configurations did not reach their true optimal performance.

Finally, some implementation differences are difficult to isolate completely. Several architectural choices interact with one another during optimization, making it challenging to attribute performance differences to a single factor in isolation. Future work could therefore explore more controlled experiments targeting individual implementation components separately.

Despite these limitations, the experiments consistently demonstrate that implementation-level differences can substantially affect both evaluation results and computational behavior, highlighting the importance of careful experimental alignment in recommender system research.

Chapter 9

Conclusion

This work investigated the differences between the official GRU4Rec implementation and the RecPack reimplementation in order to assess whether the performance discrepancies reported in previous comparison studies originate from the underlying recommendation model itself or from implementation-level differences.

Through detailed implementation analysis and controlled experimentation, multiple important discrepancies were identified between both implementations. These include differences in embedding configuration, dropout behavior, ranking strategy, batching procedure, scoring implementation, and hyperparameter optimization methodology. The experiments demonstrate that several of these differences substantially influence both recommendation quality and computational efficiency.

The ablation studies showed that enabling a no-embedding configuration significantly improves the performance of RecPack, particularly for the BPR-max objective. In addition, the experiments on ranking strategies revealed that evaluation methodology itself can strongly affect reported results under extreme tie conditions, potentially leading to misleading hyperparameter selection and unfair comparisons. Decoupled dropout parameters were found to have a smaller direct effect on recommendation quality, but improve configurability and alignment with the original GRU4Rec implementation.

The final experiments on additional datasets demonstrated that the performance gap between both implementations is substantially smaller than suggested by the original comparison study. After introducing support for missing configurations, aligning evaluation procedures, and independently optimizing both implementations, RecPack achieved performance that was competitive with the official implementation on several datasets, particularly when using the Cross-Entropy objective.

These findings indicate that a considerable portion of the originally reported discrepancy can be attributed to implementation differences and experimental methodology rather than to fundamental differences in recommendation quality. In particular, the results highlight the importance of independently tuning competing implementations before drawing conclusions regarding their relative performance.

At the same time, the experiments revealed remaining limitations in the RecPack implementation, especially on larger and more challenging datasets. The largest performance differences were observed on Taobao, where the official implementation consistently achieved substantially better results under both optimization objectives. Runtime analysis further showed that RecPack incurs additional computational overhead due to its full-catalog scoring strategy and static batching procedure. These

implementation choices become increasingly costly as dataset scale and sparsity grow.

Despite these limitations, RecPack remains a valuable framework for experimentation and research. Its modular PyTorch-based implementation makes it considerably easier to extend, modify, and integrate with modern deep learning workflows than the original Theano-based GRU4Rec implementation. After aligning important implementation details and enabling missing configurations, RecPack is capable of achieving competitive recommendation quality on a variety of datasets while maintaining a more accessible development environment.

For large-scale recommendation scenarios where runtime efficiency and scalability are critical, the original GRU4Rec implementation remains preferable due to its optimized training procedures. However, for experimentation, education, and rapid prototyping, RecPack provides a flexible and practical alternative without sacrificing a substantial amount of recommendation quality in most settings.

Overall, this work demonstrates that fair comparison of recommender system implementations requires more than reproducing a common high-level architecture. Implementation details, supported configurations, evaluation procedures, and hyperparameter optimization can all substantially influence reported results. Consequently, reproducible recommender systems research requires careful implementation alignment and transparent experimental methodology in addition to publicly available source code.

Future work could extend this analysis to additional GRU4Rec reimplementations and other session-based recommendation architectures in order to further investigate the broader impact of implementation-level differences on reproducibility, evaluation fairness, and computational efficiency.

Appendices

Appendix A

Best Hyperparameter Configurations

Table A.1: Best hyperparameter configurations for the RecPack implementation.

Dataset	Loss	Hidden	Batch	LR	DropE	DropH	Mom.	Comp.	BPR Reg.	Final Act.
YooChoose	XE	160	32	0.145	0.20	0.65	0.00	0	–	Softmax
YooChoose	BPR-max	384	32	0.065	0.20	0.35	0.75	0	1.55	ELU-1
LastFM1K	XE	416	192	0.155	0.20	0.70	0.50	0	–	Softmax
LastFM1K	BPR-max	384	224	0.035	0.05	0.15	0.40	0	1.10	ELU-1
Instacart	XE	512	48	0.070	0.25	0.70	0.75	64	–	Softmax
Instacart	BPR-max	224	32	0.030	0.35	0.30	0.40	128	1.35	ELU-0.5
MovieLens	XE	224	96	0.070	0.40	0.40	0.00	0	–	Softmax
MovieLens	BPR-max	448	32	0.135	0.30	0.40	0.65	0	0.05	Linear
Taobao	XE	448	144	0.070	0.50	0.35	0.40	512	–	Softmax
Taobao	BPR-max	416	32	0.090	0.15	0.45	0.20	0	0.75	ELU-0.5

Table A.2: Best hyperparameter configurations for the official GRU4Rec implementation.

Dataset	Loss	Hidden	Batch	LR	Drope	Dropt	Mom.	BPR Reg.	Final Act.	Sample α
YooChoose	XE	160	208	0.035	0.05	0.45	0.45	–	Softmax	0.0
YooChoose	BPR-max	256	160	0.050	0.50	0.40	0.45	0.05	Linear	0.1
LastFMUK	XE	160	80	0.045	0.25	0.20	0.15	–	Softmax	0.0
LastFMUK	BPR-max	128	48	0.090	0.30	0.05	0.45	0.45	Linear	0.5
Instacart	XE	160	160	0.050	0.40	0.30	0.15	–	Softmax	0.0
Instacart	BPR-max	288	128	0.050	0.45	0.30	0.05	0.95	ELU-1	0.2
Movielens	XE	64	112	0.065	0.10	0.25	0.05	–	Softmax	0.0
Movielens	BPR-max	64	64	0.040	0.25	0.50	0.20	0.25	ELU-1	0.3
Taobao	XE	128	176	0.035	0.35	0.35	0.60	–	Softmax	0.0
Taobao	BPR-max	96	80	0.065	0.30	0.05	0.30	1.35	Linear	0.5

Dataset	Official XE	RecPack XE	Δ XE	Official BPR	RecPack BPR	Δ BPR
YooChoose	0.6657	0.6480	-0.0177	0.6694	0.6327	-0.0367
Instacart	0.1800	0.1705	-0.0095	0.1856	0.1627	-0.0229
MovieLens	0.1078	0.1060	-0.0018	0.1165	0.0867	-0.0298
LastFM1K	0.4425	0.3989	-0.0436	0.4629	0.4027	-0.0602
Taobao	0.2128	0.1112	-0.1016	0.2063	0.1109	-0.0954

References

- [1] Last.fm 1k dataset. <https://github.com/jlieth/lastfm-dataset-1K-extended/>.
- [2] Movielens 100k dataset. <https://grouplens.org/datasets/movielens/>.
- [3] Taobao user behavior dataset. <https://www.kaggle.com/datasets/marwa80/userbehavior>.
- [4] Rakesh Agrawal, Tomasz Imieliński, and Arun Swami. Mining association rules between sets of items in large databases. In *Proceedings of the 1993 ACM SIGMOD International Conference on Management of Data*, SIGMOD '93, page 207–216, New York, NY, USA, 1993. Association for Computing Machinery.
- [5] David Ben-Shimon, Alexander Tsikinovsky, Michael Friedmann, Bracha Shapira, Lior Rokach, and Johannes Hoerle. Recsys challenge 2015 and the yoochoose dataset. In *Proceedings of the 9th ACM Conference on Recommender Systems, RecSys 2015, Vienna, Austria, September 16-20, 2015*, pages 357–358, 2015.
- [6] Kyunghyun Cho, Bart van Merriënboer, Dzmitry Bahdanau, and Yoshua Bengio. On the properties of neural machine translation: Encoder-decoder approaches, 2014.
- [7] Diginetica. Cikm cup 2016. https://competitions.codalab.org/competitions/11161#learn_the_details-data2, 2016.
- [8] The Theano Development Team et al. Theano: A python framework for fast computation of mathematical expressions, 2016.
- [9] Xavier Glorot and Yoshua Bengio. Understanding the difficulty of training deep feedforward neural networks. In *Proceedings of the Thirteenth International Conference on Artificial Intelligence and Statistics*, pages 249–256, 2010.
- [10] Balázs Hidasi and Ádám Tibor Czapp. The effect of third party implementations on reproducibility. In *Proceedings of the 17th ACM Conference on Recommender Systems, RecSys '23*, page 272–282. ACM, September 2023.
- [11] Balázs Hidasi and Alexandros Karatzoglou. Recurrent neural networks with top-k gains for session-based recommendations. In *Proceedings of the 27th ACM International Conference on Information and Knowledge Management, CIKM '18*, page 843–852. ACM, October 2018.

-
- [12] Balázs Hidasi, Alexandros Karatzoglou, Linas Baltrunas, and Domonkos Tikk. Session-based recommendations with recurrent neural networks, 2016.
 - [13] Instacart. The Instacart Online Grocery Shopping Dataset. <https://www.instacart.com/datasets/grocery-shopping-2017>, 2017.
 - [14] Dietmar Jannach, Massimo Quadrana, and Paolo Cremonesi. Session-based recommender systems. In Francesco Ricci, Lior Rokach, and Bracha Shapira, editors, *Recommender Systems Handbook*, pages 301–334. Springer US, 2022.
 - [15] Malte Ludewig and Dietmar Jannach. Evaluation of session-based recommendation algorithms. In *User Modeling, Adaptation and Personalization*, 2018.
 - [16] Malte Ludewig and Dietmar Jannach. Evaluation of session-based recommendation algorithms. *User Modeling and User-Adapted Interaction*, 28, 12 2018.
 - [17] Lien Michiels, Robin Verachtert, and Bart Goethals. Recpack: An(other) experimentation toolkit for top-n recommendation using implicit feedback data. In *Proceedings of the 16th ACM Conference on Recommender Systems*, RecSys ’22, page 648–651, New York, NY, USA, 2022. Association for Computing Machinery.
 - [18] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Köpf, Edward Yang, Zach DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. Pytorch: An imperative style, high-performance deep learning library, 2019.
 - [19] Joelle Pineau, Philippe Vincent-Lamarre, Koustuv Sinha, Vincent Larivière, Alina Beygelzimer, Florence d’Alché Buc, Emily Fox, and Hugo Larochelle. Improving reproducibility in machine learning research (a report from the neurips 2019 reproducibility program), 2020.
 - [20] REES46 Marketing Platform. ecommerce behavior data from multi category store, 2020.
 - [21] Massimo Quadrana, Paolo Cremonesi, and Dietmar Jannach. Sequence-aware recommender systems. *ACM Computing Surveys*, 51(4):1–36, 2018.
 - [22] Steffen Rendle, Christoph Freudenthaler, Zeno Gantner, and Lars Schmidt-Thieme. Bpr: Bayesian personalized ranking from implicit feedback. In *Proceedings of the twenty-fifth conference on uncertainty in artificial intelligence*, pages 452–461, 2009.
 - [23] Retailrocket. Retailrocket recommender system dataset, 2017.
 - [24] Francesco Ricci, Lior Rokach, and Bracha Shapira. *Recommender Systems Handbook*. Springer, 2nd edition, 2015.
 - [25] Badrul Sarwar, George Karypis, Joseph Konstan, and John Riedl. Item-based collaborative filtering recommendation algorithms. *Proceedings of ACM World Wide Web Conference*, 1, 08 2001.

- [26] Guy Shani, Ronen I. Brafman, and David Heckerman. An mdp-based recommender system, 2015.
- [27] Zhu Sun, Hui Fang, Jie Yang, Xinghua Qu, Hongyang Liu, Di Yu, Yew-Soon Ong, and Jie Zhang. Daisyrec 2.0: Benchmarking recommendation for rigorous evaluation. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 45(7):8206–8226, July 2023.
- [28] Jacopo Tagliabue, Ciro Greco, Jean-Francis Roy, Bingqing Yu, Patrick John Chia, Federico Bianchi, and Giovanni Cassani. Sigir 2021 e-commerce workshop data challenge, 2021.
- [29] Shuai Zhang, Lina Yao, Aixin Sun, and Yi Tay. Deep learning based recommender system: A survey and new perspectives. *ACM Computing Surveys*, 52(1):1–38, 2019.